Lecture 00: Introduction



TMA3084 Software Engineering Lab

	Topic
☑	Course Coordinator/Lecturer & Lab Demos
☑	Prerequisite Subjects
☑	Course Synopsis
☑	Learning Outcomes
☑	Course Plan
☑	Assessments
☑	References

Coordinator / Lecturer



Ts. Nurfauza Jali

jnurfauza@unimas.my

Level 3,FCSIT

Ext: 3814

Lab Demos



MD. Aminul Islam Rakib

24020247@siswa.unimas.my



Raynold Kabai

100853@siswa.unimas.my

Prerequisites Subject

TMF2243

Object Oriented Software Engineering

Course Synopsis

- This course provides students with the necessary skills for students to develop a mobile application product as a team.
- Students will be introduced to the Team Software Process (TSP);
  1. an integrated framework that guides development teams in producing high-quality software-intensive systems.
  2. It also consists of an introduction to the methods presented in the TSP which are;
     - ❑ how to establish roles;
     - ❑ How to conceive, design, and plan a project;
     - ❑ and how to track and report on software project's progress.
  3. Produce a clear and complete Software Requirement Specification (SRS), Software Design Specification(SDS) and furthermore experience in handling test plan.

Learning Outcomes

By the end of this course students will be able to:

- Experiment the use of **software engineering methodology** in a **team environment** using real-world application, whereby each person will perform their own roles.
- Organize the project based on the software engineering process and practice including **software development life cycle**, standards of **analysis and design** and aspects of **management** for software engineering project.
- Demonstrate the advantages of using several **Computer Aided Software Engineering Tools (CASE Tools)** in major phases of software development.
- Work as a team in **solving complex problem** while giving appropriate **justification** and able to **present solution clearly** and confidently.

Course Plan: TMA3084
Software Engineering Lab (Sem 1, 2026/2027)

BUILD TEAMS • DESIGN SOLUTIONS • DEVELOP TOGETHER • MAKE AN IMPACT

From Ideas to Real Solutions

PEOPLE
PROCESS
PRODUCT
SUCCESS

PLAN TOGETHER
DEVELOP SOLUTIONS
TEST THOROUGHLY
DOCUMENT PROGRESS
PRESENT YOUR WORK
ACHIEVE MORE

"Good Software is a Team Effort"

Week	Date	Lecture Topic	Labs	Activities	Assessment %
1	21 – 27 Sept 2026	Lecture 0: Introduction Lecture 1: Team Software Process (TSP) and Agile Methodology Overview	–	Team formation and Project Selection	Project Proposal & Planning (10%)
2	28 Sept – 4 Oct 2026	Lecture 2: The Logic of Team Software Process (TSP)	–	–	–
3	5 – 11 Oct 2026 *10 Oct 2026 (Saturday)* *TTT Sarawak Birthday*	Lecture 3: Launching a Team Project	**Lab 01**	–	–
4	12 – 18 Oct 2026	Lecture 4: The Development Strategy	**Lab 02**	–	–
5	19 – 25 Oct 2026	Lecture 5: The Development Plan	**Lab 03**	Proposal Presentation	Requirement Specifications (SRS) (15%)
6	26 – 1 Nov 2026	Lecture 6: Defining the Requirements	**Lab 04**	–	–
7	2 – 8 Nov 2026	Lecture 7: Designing with Teams	**Lab 05**	–	–
8	**MidTerm-Sem Break (9 – 15 Nov 2026)**				
9	16 – 22 Nov 2026	Lecture 8: Product Implementation	**Lab 06**	Project Presentation (Initial)	–
10	23 – 29 Nov 2026	Lecture 9: Integration and System Testing	**Lab 07**	–	–
11	30 Nov – 6 Dec 2026	Lecture 10: The Postmortem	**Lab 08**	–	Design and Implementation Specification (SDS) (25%)
12	7 – 13 Dec 2026 *25 Dec 2026 (Friday)* *Christmas*	Discussion	–	–	–
14	14 – 20 Dec 2026	–	–	–	Testing and Release (10%)
15	21 – 27 Dec 2026	–	–	Project Report Submission	–
16	28 Dec 2026 – 3 Jan 2027 *1 Jan 2026 (Friday)* *New Year*	–	–	Project Presentation (Final)	Presentation (10%)

Study Week (4 – 10 Jan 2027) *17 & 18 Feb 2026 (Chinese New Year)*

Examination Week (11 Jan – 31 Jan 2027)

Semester Break (1 Feb – 28 Feb 2027)



Project Deliverables

Phase 1
PLANNING & DEVELOPMENT

Proposal Submission	Proposal Presentation	Initial Report	Demo 1
Week 4	Week 5	Week 9	Week 9
9 Oct 2026	19–23 Oct 2026	16 Nov 2026	16–20 Nov 2026

Phase 2
FINALIZATION & CLOSURE

Final Report	Final Demo 2	e-Archive
Week 13	Week 14	Week 15
24 Dec 2026	28–31 Dec 2026	Before 10 Jan 2027

Final Project (60%)
PLAN · DESIGN · BUILD · DELIVER

Project Proposal & Planning	Requirement Specifications	Design and Implementation Specification	Testing & Release
10 marks	15 marks	25 marks	10 marks

Oral Commitment (10%)
COMMUNICATE & ENGAGE

Presentation
10 marks

Practical Examination (30%)
COLLABORATE · APPLY · PERFORM

Peer Evaluation	Individual Evaluation (Personal log, participation, presentations, etc.)	Team Evaluation (foundation and functioning)
10 marks	10 marks	10 marks

TMA3084 Software Engineering Lab

Group Project Assessment



Flutter + Dart Mobile-first app

Real client
Needs, feedback, sign-off

AI feature
Useful, not decorative

Evidence trail
GitHub, tests, reports

Main idea: do not just build an app — prove that it solves a real problem.

The Mission & The Standard

What the project must achieve in plain words.



Your mission

Build a mobile app that a real client can actually use.

The standard

Useful, usable, maintainable, and supported by evidence.

- Solve a genuine client problem, not a made-up one.
- Use Flutter/Dart and keep the app mobile-first.
- Include at least one meaningful AI-driven feature.
- Submit documents, tests, GitHub evidence, and client sign-off.

Project Journey (At a Glance)

1. Team formation and Project selection
2. Proposal Presentation
3. Development & Implementation
4. Testing and Release
5. Project Report Submission
6. Presentation (Final)

Study Week (4 – 10 Jan 2027) | Examination Week (11 Jan – 31 Jan 2027) | Semester Break (1 Feb – 28 Feb 2027)

A working app is necessary. A clear evidence trail is what proves the work.

The Launch Sequence

Move through the project in small, visible steps.

0
Sprint Zero
Team, client, repo, tools

1
Proposal + SRS
Problem, scope, requirements

2
Design + Prototype
SDD, UI flow, database, AI plan

3
Build + Test
Integrated app, PRs, logs, fixes

4
Final Demo
Client sign-off, report, archive

Do not wait until the end to show progress.
Each milestone must leave behind evidence.
If scope becomes too big, reduce features early — not during final week.

Progress is easier to manage when every step produces something visible.

Squad Accountability Matrix

Each role owns a part of the project and the evidence for it.

Role	Main responsibility	Evidence to show
Project Manager	Keeps schedule, meetings, risks, and scope under control.	Meeting notes, sprint board, risk log
Client / Requirements Lead	Talks to client and confirms what the app must do.	SRS, interview notes, sign-off
UI / UX + Design Lead	Turns requirements into screens, flow, architecture, and database design.	SDD, wireframes, ERD/UML
Implementation + AI Lead	Coordinates coding, AI integration, and pull requests.	PRs, commits, AI audit log
QA + CI/CD Lead	Plans tests and checks quality before demo.	Test cases, CI logs, defect log

Roles are not titles. A role means someone is responsible for making that evidence exist.

TeamTopology: Match the Rhythm to the Team

Where the team is located affects communication rhythm.

Local Core

Most members meet on campus

Best rhythm:

Short stand-up + weekly planning

Hybrid Team

Some members remote or busy

Best rhythm:

Sync updates + fixed review slot

Satellite Support

Client or specialist not always available

Best rhythm:

Prepared questions + recorded decisions

- Pick one normal communication rhythm and protect it.
- Use a shared board so no task is hidden.
- Raise blockers early; silent delays hurt the whole team.

Good coordination beats last-minute hero work.

PLAN TOGETHER

DEVELOP SOLUTIONS

TEST THOROUGHL

DOCUMENT PROGRESS

PRESENT YOUR WORK

ACHIEVE MORE

"Good Software is a Team Effort"

Trust but Verify: The AI Audit Trail

Use AI tools responsibly and keep a clear record.

Prompt / request
What did you ask the AI?

→

AI output
What did it produce?

→

Human review
What did the team check or change?

→

Project link
Which issue, PR, or file used it?

→

Compliance check
Is it safe and allowed?

Minimum log fields: date, task, AI tool, prompt summary, output summary, human changes, link to PR or document.
Do not submit AI output that the team cannot explain.

AI can help you move faster, but the team remains responsible for the final work.

The Client Partnership Engine

Keep the client involved from discovery until final sign-off.

1. Discover
Interview the client
and learn the real pain point

2. Confirm
Write requirements
and ask the client to agree

3. Build
Show small progress
through prototype and demos

4. Refine
Use feedback
to improve the next version

- Schedule weekly or fortnnightly check-ins.
- Record client decisions in meeting notes or messages.
- Avoid silent assumptions — ask, confirm, then build.
- Collect sign-off when the client accepts key deliverables.



Your client is a project partner, not only an examiner.

Meaningful AI Integration

Use AI to improve the user experience, not just to look trendy.

Virtual assistant
Answers common user questions.

Computer vision / OCR
Reads images, receipts, forms, or IPs.

Prediction
Forecasts demand, trends, or risk.

Recommendation
Suggests useful options based on user data.

FOLLOW THESE STEPS

- Start from the user problem, then choose the AI feature.
- Explain what data goes in and what answer comes out.
- Test the AI result with real examples from your project.
- Always record AI usage in the audit trail.



Ask: “Does this AI feature make the app more useful for the client?”

LEARN
PRACTICE
COLLABORATE
GROW

Professional Integrity & Sprint Zero

Start clean, work honestly, and set up the team properly.

Warning

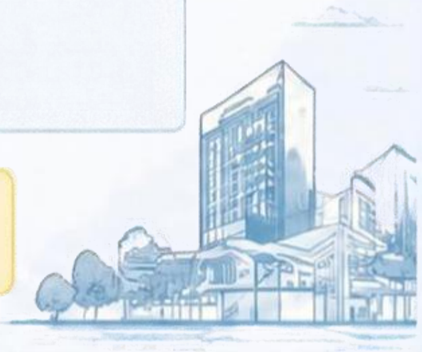
Taking credit for someone else's work is plagiarism.

AI is allowed as support, but the team must review, understand, and disclose its use.

Sprint Zero: Key Setup Actions

- Form the team and agree on roles.
- Secure a real client and project scope.
- Create GitHub repo, branches, issue board, and README.
- Agree on communication rules and weekly meeting time.

Sprint Zero is preparation. It prevents chaos later.



The Standard Engineering Stack

Keep the technology choices clear and explainable.

UI layer
Flutter screens, navigation, responsive design

App logic layer
Dart services, state management, validation

Data layer
Firebase / Supabase / REST API, database

Intelligence layer
AI API or local model, prompt/spec control

Quality layer
GitHub Actions, tests, logs, release archive

Use simple technology choices that your team can maintain and explain during demo.



Continuous Quality with GitHub Actions

Use the repository as the control centre for quality.

Feature branch
Small, focused changes

→

Pull request
Peer review before merge

→

Automated checks
Lint, unit/widget tests, build

→

Main branch
Only stable work is merged

QUALITY CHECKPOINTS

- Every feature should enter through a pull request.
- Use reviews to catch problems early.
- Show CI logs as evidence of quality checks.
- Track defects in issues or defect logs, not in memory.



A clean GitHub history tells the story of the team's work.

The Traceability Engine

Connect every feature from client need to final test.



Client Need
What problem are we solving?

→

SRS
What must the system do?

→

SDD
How will it be designed?

→

GitHub PR
Where was it built?

→

Test Log
How was it verified?

→

Demo
How did the client see it?

- Number requirements so they can be traced.
- Link issues, commits, pull requests, and test cases back to requirements.
- Keep the evidence updated while building, not after everything is done.

If the trace is broken, the team may struggle to prove the feature is complete.

Collaboration & Evidence Modes

Choose working habits that match your team setup.

Co-located / Local

- Face-to-face discovery
- Whiteboard planning
- Quick pair-programming

Evidence

- Meeting photos or minutes
- Signed notes
- GitHub commits

Scattered / Hybrid

- Sync task tracking
- Online design reviews
- Remote client confirmation

Evidence

- GitHub Projects
- Online sign-offs
- Recorded decisions

No merge to main without a pull request and at least one peer approval.

Testing & Definition of Done

A feature is not done just because it runs once.

Code
merged by PR, no obvious errors

Requirements
Matches agreed SRS item

Design
Fits SDD / architecture

Testing
Unit, integration, or UAT evidence

User docs
Manual or setup guide updated

Client
Shown or accepted by client when needed

Done = built, reviewed, tested, documented, and traceable.

From Random Coding to Spec-Driven “Vibe Coding”

Let AI help with syntax, but let engineering guide the system.

Old habit

- Start coding immediately
- Copy-paste fixes
- No clear requirement link
- Testing at the very end

Better habit

- Write a small spec first
- Ask AI for a draft, not the final answer
- Review and test carefully
- Commit with traceable evidence

The AI writes some syntax. The human engineers own the requirements, design, quality, and decisions.

Next action: finish Sprint Zero, confirm the client problem, and create your evidence trail from day one.

The Definition of Done

Code & Infrastructure

- ✓ GitHub repository
- ✓ Git Flow history
- ✓ Passing CI/CD Actions
- ✓ v1.0.0 APK/IPA builds

Technical Documentation

- ✓ SRS (Requirements)
- ✓ SDD (Design/Architecture)
- ✓ User Manual
- ✓ API Documentation

Audits & TSP Compliance

- ✓ LOGAI (AI prompts)
- ✓ LOGD & LOGTEST
- ✓ LOGT (Timesheets)
- ✓ Peer Evaluations

Client Assets

- ✓ Final Presentation
- ✓ Demo Video
- ✓ Signed Client Acceptance Form

BUILD TEAMS | DESIGN SOLUTIONS | DEVELOP TOGETHER | MAKE AN IMPACT

A successful project satisfies client needs, ensures exceptional usability, and guarantees maintainlability over its 3-year life

Professional Integrity & Sprint Zero

BUILD TEAMS • DESIGN SOLUTIONS • DEVELOP TOGETHER • MAKE AN IMPACT

From Ideas to Real Solutions

PEOPLE
PROCESS
PRODUCT
SUCCESS

The Warning

> **"Taking credit for someone else's work... is plagiarism."**

Using AI is a mandatory requirement.
Using AI without documentation in LOGAI is severe academic misconduct.

Sprint Zero Initiates Now

1. **Form the Team**
   (Lock in 5 members, submit FORM-SI)
2. **Secure the Client**
   (Conduct discovery interviews)
3. **Initialise the Canvas**
   (Draft charter, set up GitHub Org)

COURSE PLAN HIGHLIGHTS

- Team formation and Project Selection
- Project Proposal & Planning (10%)
- Project Presentation (Initial)
- Design and Implementation Specificisation (SRS) (25%)
- Testing and Release (10%)

- Study Week (4 – 10 Jan 2027) *17 & 18 Feb 2026 (Chinese New Year)*
- Examination Week (11 Jan – 31 Jan 2027)
- Semester Break (1 Feb – 28 Feb 2027)



LEARN
PRACTICE
COLLABORATE
GROW

> CLOCK STARTS NOW. Final submission: 02 January 2027. Build something that matters.

TMA3084 Software Engineering Lab

Questions & Answers



Flutter + Dart Mobile-first app

Real client
Needs, feedback, sign-off

AI feature
Useful, not decorative

Evidence trail
GitHub, tests, reports

Main idea: do not just build an app — prove that it solves a real problem.